pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Case Study: Where the Classifier and the Transformer Disagree

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Three models, one comparison

TF-IDF (day 3)

Fitted on 3,650 labelled sections; validation macro F1 0.9271.

Embedding (day 7)

MiniLM's 384 numbers, then a logistic regression; 0.8882.

Zero-shot (day 7)

No labelled section, eight descriptions; 0.5951.

Fitted on 2019 to 2023, chosen on 741 sections of 2024; the 735 of 2025 opened once, in 10.6.

Real documents, described

An 8-K section's label is the filer's item; a model's output is the item it assigns

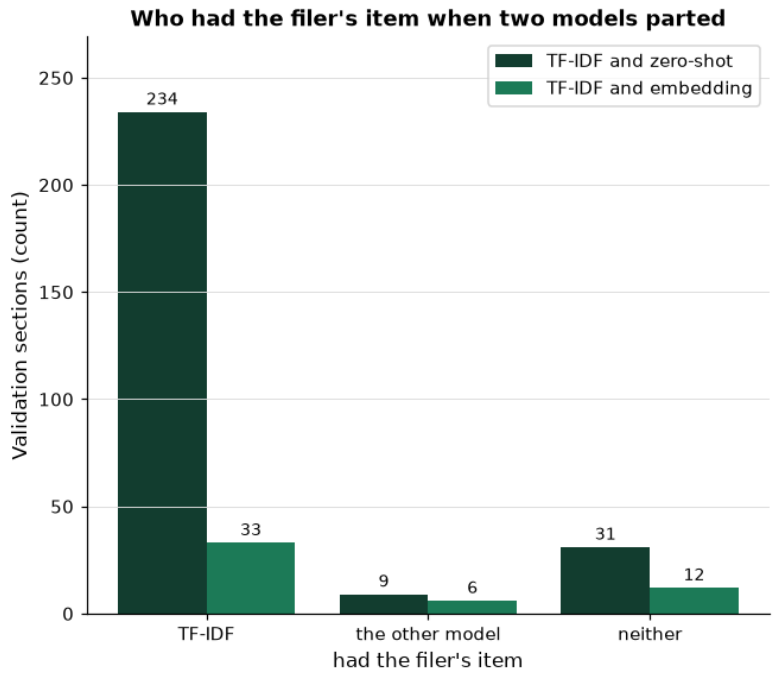
Allowed names	Never called
the item the model assigns	a default signal
the model's score for an item	a distress flag or a warning
the filer's item	a credit event or credit view

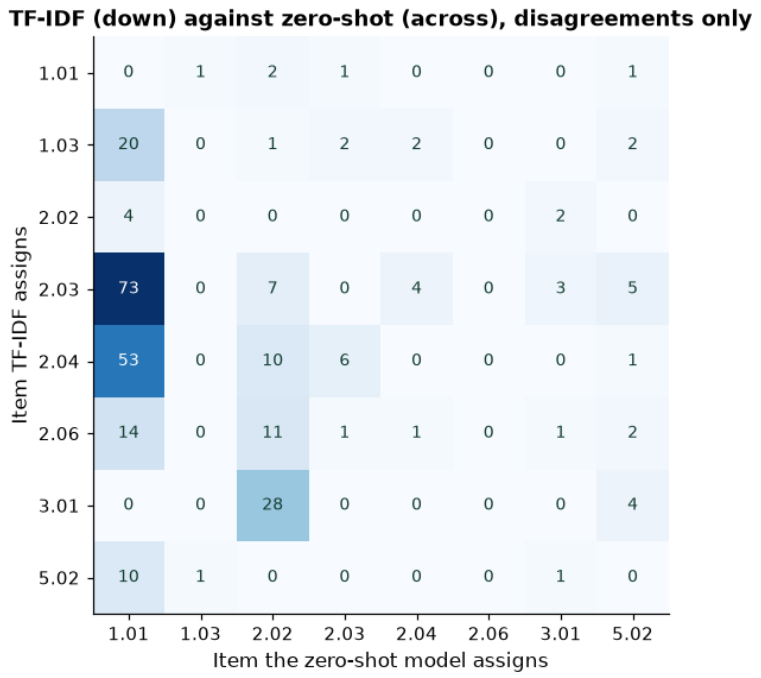
Item 5.02 sections name people: counted by item only; no 5.02 disagreement is read.

How disagreements are found

The problem	Read the rows behind an agreement share, not just the share
The compare	Two columns of labels as text; keep the rows where they differ
Who matched	A column: the model that matched the filer's item, or neither
What you control	The two models and the column holding the truth
The limit	Different words to go on; not which model is right in general

Where two models part





274 of 741: TF-IDF and zero-shot disagree

51 of 741: TF-IDF and embedding

When TF-IDF and zero-shot part, the filer's item is TF-IDF's 234 times, zero-shot's 9 times, neither's 31. Zero-shot assigns 1.01 to 174.

A disagreement shows where two models had different words to go on.

Ten disagreements, read

#	Filer	TF-IDF	Zero-shot	Words	TF-IDF's first term
1	1.03	1.03	1.01	300	dip
2	2.04	2.04	1.01	300	payment
3	2.03	2.03	1.01	28	item 2.03
4	2.04	2.04	1.01	300	default
5	2.06	2.06	2.02	101	charge
6	2.03	2.03	1.01	60	item 1.01
7	1.01	2.03	1.01	154	the note
8	2.03	2.03	1.01	102	the note
9	2.04	2.04	2.03	221	default
10	2.04	2.04	1.01	263	default

What each model went on

TF-IDF: the terms with the largest parts, its TF-IDF value times the item's coefficient. Zero-shot: the first 120 words, against eight descriptions.

Disagreements name another item in the body: 0.3540, against 0.1754 of all sections.

The case study question

On sections none of the three was chosen on, how often does each assign the filer's item, item by item? Are the differences larger than the noise of which sections were filed?

Fix

the models as chosen

Open

the 735 test sections once

Score

macro F1, recall per item

Interval

paired, 1,000 resamples

An item is a heading a filer chose; no output here says anything about a company.

What each model reads

TF-IDF

Every word of the stored body, up to 300.

MiniLM

The first 256 word pieces, then 384 numbers.

Zero-shot

The first 120 words, against eight sentences.

0.6544

of the test sections longer than 120 words

0.2578

cut at 300 words by the course's tool

The descriptions are day 7's, word for word: never reworded after a result.

Opened once, checked first

735

test sections, 708 filings

25 of 25

same zero-shot item live and from the file

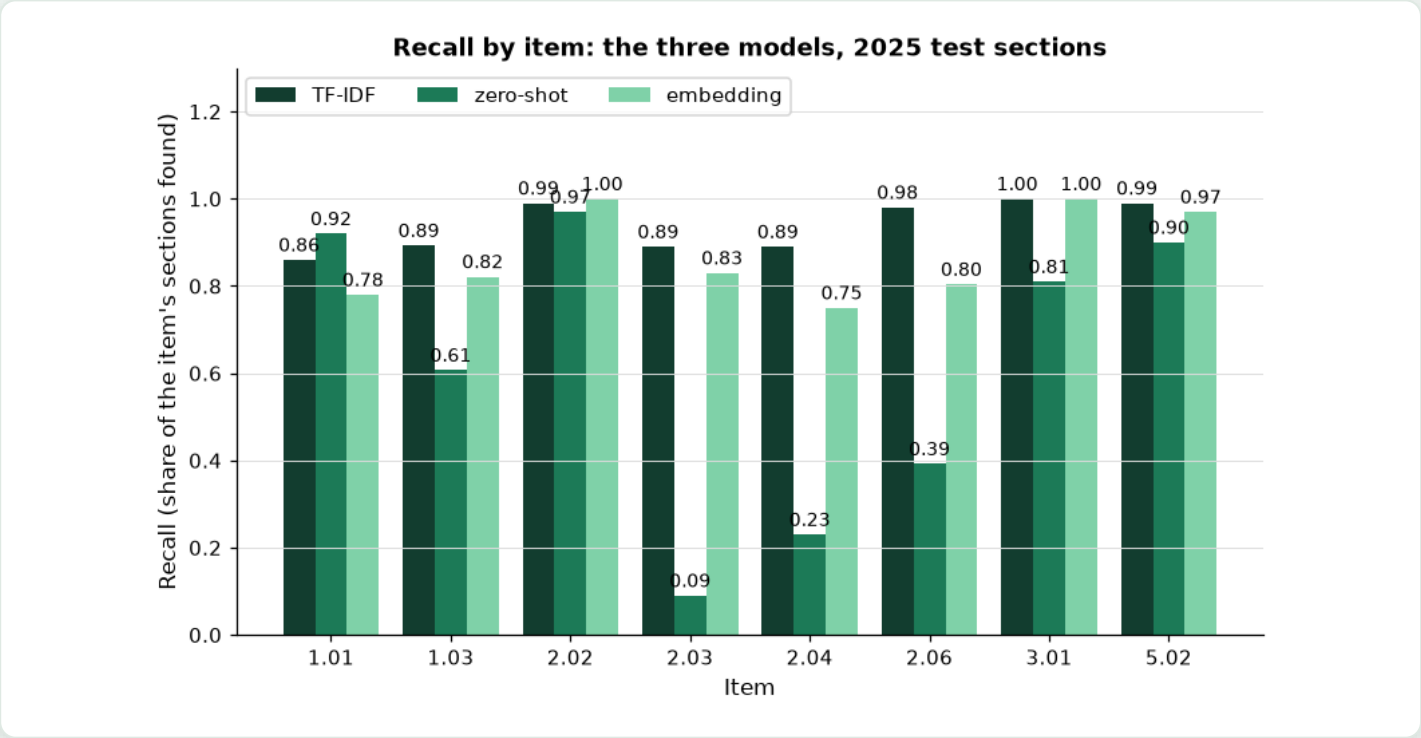
True

25 live embeddings within 1e-6

```
tolerance = 1e-4 if torch.get_num_threads() == 1 else 1e-3
# live zero-shot scores equal the committed file's: True; no live score is printed
```

Filed 2025-01-02 to 2025-12-31: 100 per item, but 1.03 (84) and 2.06 (51).

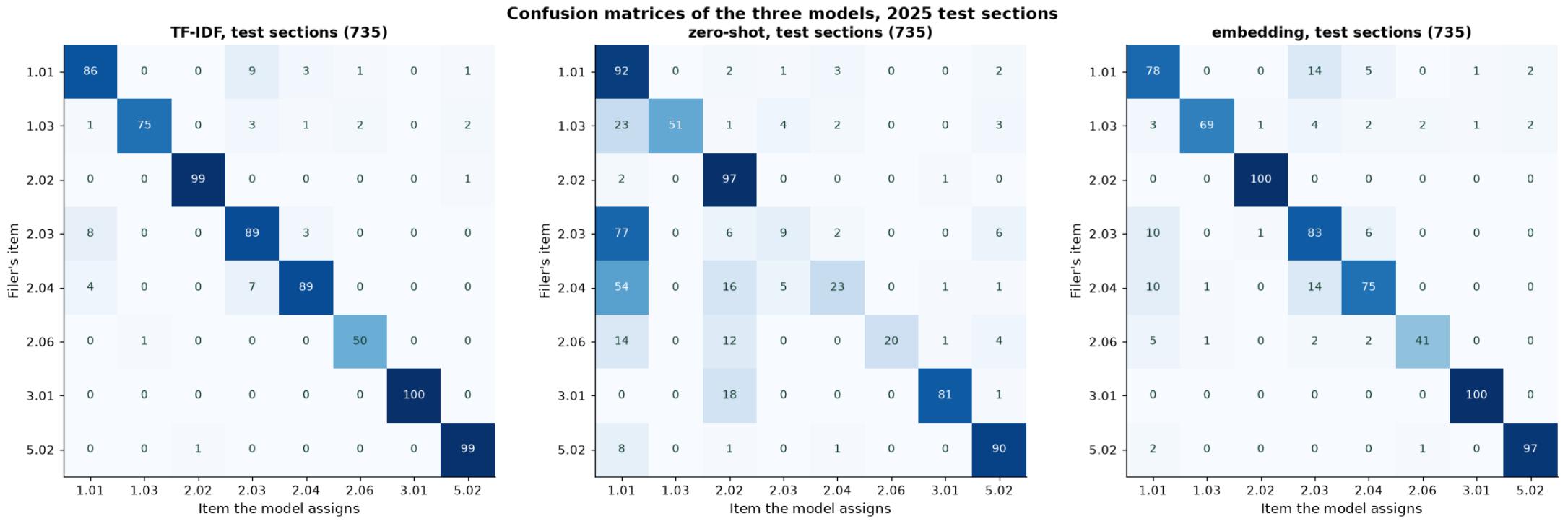
Three models on the 2025 test sections



	TF-IDF	Zero-shot	Embedding
recall 1.03	0.8929	0.6071	0.8214
recall 2.03	0.8900	0.0900	0.8300
recall 2.04	0.8900	0.2300	0.7500
recall 2.06	0.9804	0.3922	0.8039
macro F1	0.9366	0.6044	0.8754

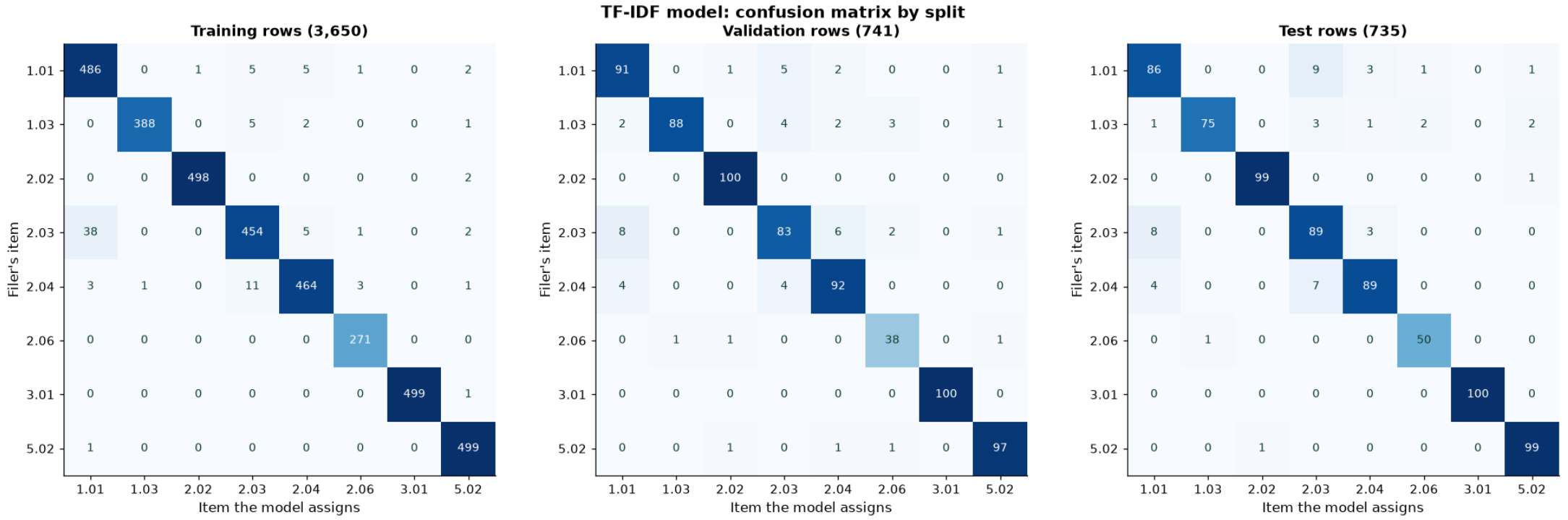
The majority item, 1.01 for every section: macro F1 0.0299, accuracy 0.1361. Each column is the item the model assigns, scored against the filer's item.

Confusion matrices on the test rows



The zero-shot model's dark column is 1.01. The same picture as on validation.

The TF-IDF model, split by split

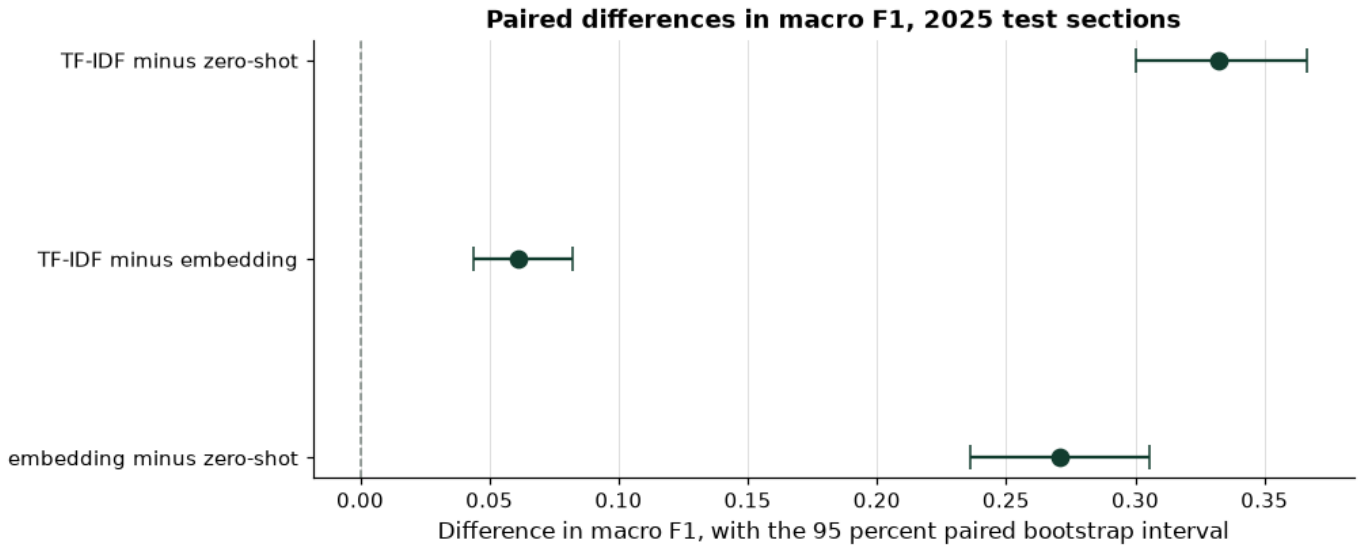


Training rows: the model was fitted on them. Test rows: opened once, after the models were chosen on validation.

How a paired bootstrap works

The problem	Is a difference between two models bigger than the luck of which sections were filed?
Resample	Draw the scored sections with replacement, the same number as before
Paired	Score both models on the same drawn sections; record the difference in macro F1
Repeat	1,000 times; the 2.5th and 97.5th percentiles are the 95 percent interval
What it says	Describes the sections scored, not a refit on other data

Paired bootstrap intervals



Pair	Value	Lower	Upper
TF-IDF minus zero-shot	0.3322	0.3003	0.3664
TF-IDF minus embedding	0.0612	0.0437	0.0822
Embedding minus zero-shot	0.2710	0.2360	0.3055

```
ensemblekit.bootstrap_interval(truth, positions, paired_difference(a, b), seed=SEED)
```

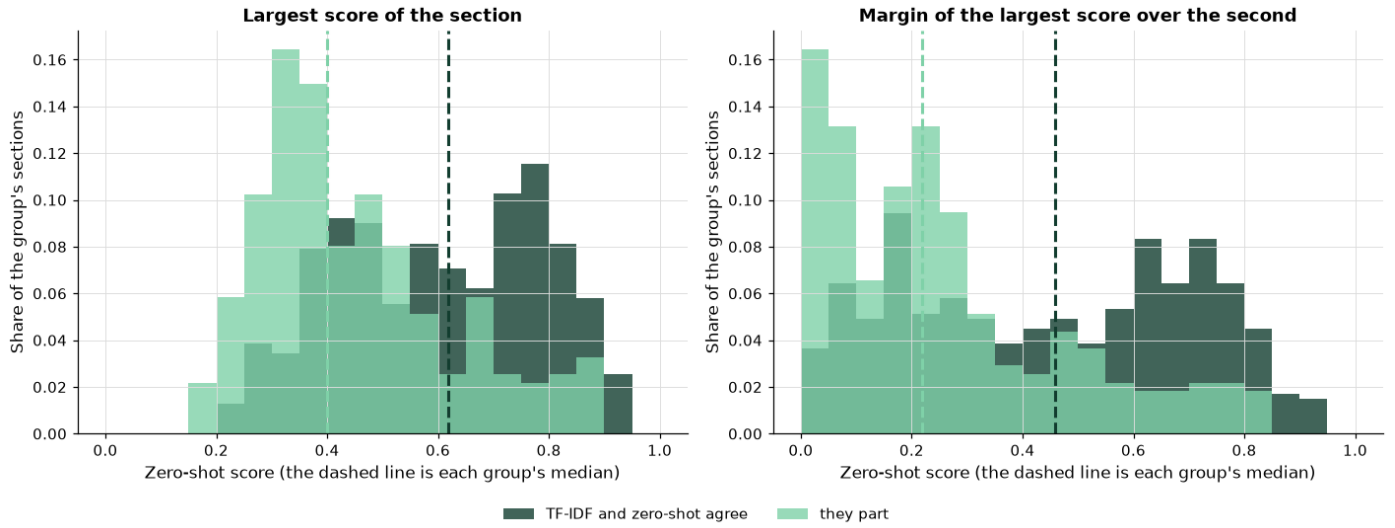
Both models scored on the same resampled rows. Intervals that hold 0: 0 of 3.

Where it fails

Sampling	100 per item per year, by the course's design
One label	147 test sections name another item
One period	fitted 2019 to 2023, chosen 2024, tested 2025
The cut	0.2578 cut at 300 words
The sample	25 recomputed live; the rest trusted

The zero-shot model assigns 77 of the 100 Item 2.03 sections to 1.01.

How sure the zero-shot model was



0.619

median largest, agree

0.400

median largest, part

0.458

median margin, agree

0.218

median margin, part

467 sections agree, 274 part. A description of two groups of scores, not a verdict: day 7's scores overlapped for hits and misses.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIFS(item range,$A2,assigned range,B$1)`	the confusion matrix: 64 of 64 cells
`=AVERAGE(L2:L9)`	macro F1: TF-IDF 0.9366
`=COUNTIFS(item range,"2.040")`	100: the text 2.04 counted
`=SUMPRODUCT(--EXACT(item range,"2.040"))`	0: exact text only

COUNTIFS compares criteria that look like numbers as numbers: use EXACT for codes like 2.1 and 2.10.

Save the day: the test rows, and week 2

```
git diff results.csv
+tfidf_logistic_balanced,test,0.9366,0.9347,0.8929,0.8900,735
+zero_shot,test,0.6044,0.6299,0.6071,0.2300,735
+embedding_logistic,test,0.8754,0.8748,0.8214,0.7500,735
```

```
git tag week2
git diff --stat week2~1 week2   # results.csv | 3 +++
```

pytest: 51 passed. In my_bondmath: git diff --stat week1 week2.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: one item against the rest: zero-shot gets 0.8844 on 2.04 (recall 0.2300); never assigning it gets 0.8639.

Code that runs is not the same as code that is right.

What you can do now

- Read where two models part, and why
- Open the test rows once, for chosen models
- Check a cached output with a live sample
- Pair the bootstrap: same rows, both models
- COUNTIFS on codes: use EXACT for look-alikes

Week 3, Day 1 The first request to an LLM